

02-741 Week 2

Generative AI for Biomedicine

Aidan Jan

September 3, 2026

Modeling Structure with Graph Neural Networks

Graphs are everywhere!

- Molecular graphs
- Gene graphs
- Brain graphs
- Web graphs
- Transportation graphs
- Social graphs

There are a few common tasks we can do on graph data:

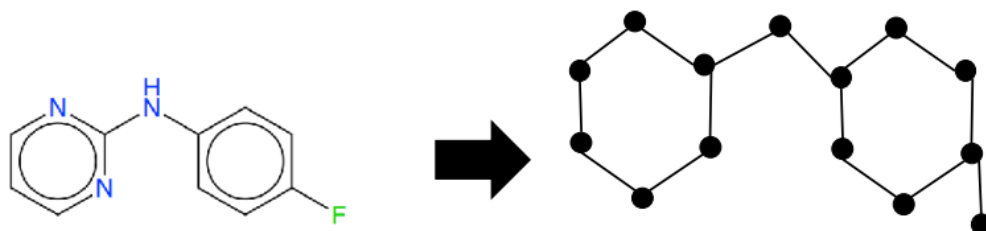
- Node-level:
 - Link prediction: is there a bond between two atoms?
 - Node classification: is this atom C or N?
 - Structure prediction: what are 3D coordinates of atoms / residues?
- Graph-level:
 - Predicting properties of a graph: Is a molecule toxic to animals?

Graph Representation

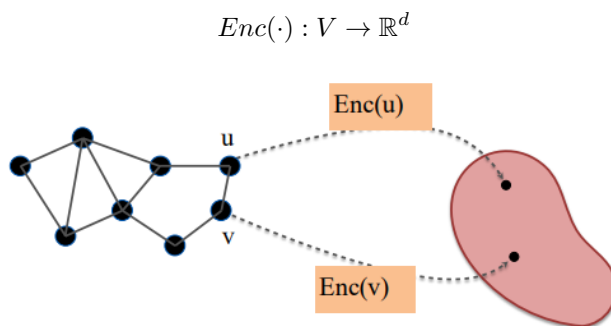
- Graph: $G = \{V, E\}$
- Nodes: $V = \{v_1, v_2, \dots, v_N\}$
- Edges: $E = \{e_1, e_2, \dots, e_M\} \subset V \times V$

Atom-level Molecule Graph

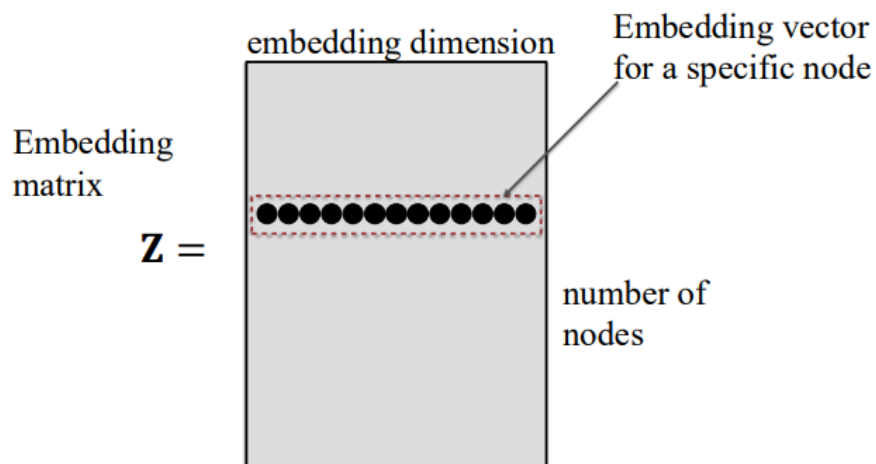
- Node: heavy (non-hydrogen) atoms
- Edge: bond



Node Embedding



A “shallow” node embedding is just a lookup table.



Graph Neural Network

- Input is a graph.
- There are many hidden GNN layers.
- Output is an embedding matrix for nodes for further downstream tasks: e.g., node property prediction.

GNN-based Foundation Models for Biomedicine

- Predicting docking
 - EquiDock
 - EquiBind
- Property prediction
 - EnviroDataNet
 - TorchMD-NET
- Modeling protein
 - GearNet
- Gene regulator network
 - DeepSEM

- Drug design
 - JT-VAE
 - GraphAF

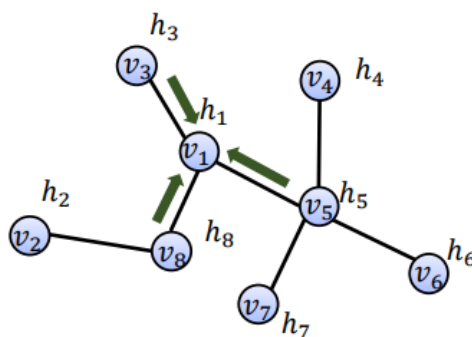
Message Passing Neural Network

Every node receives message from its neighbors, aggregate, and update

- h_i : node embedding vector. Aggregates information from its neighbors

$$h_i^{k+1} = U^k \left(\text{Aggregate}_{v_j \in N(v_i)} m_{j \rightarrow i}^k \right), \forall v_i \in V$$

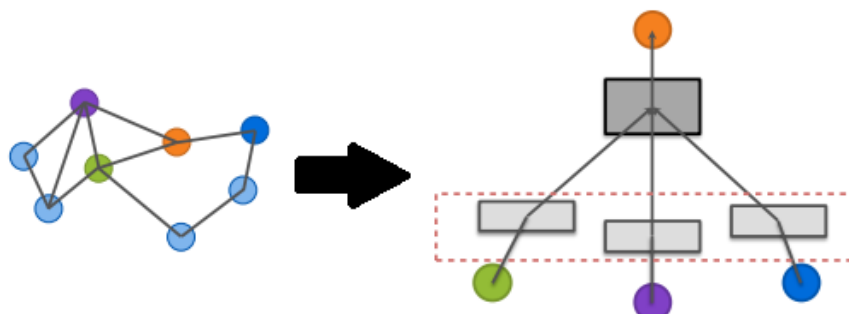
- $N(v_i)$: Neighbors of the node V_i
- $m_{j \rightarrow i}^k$: message from node j to i at k -th layer. Aggregate can be average, sum, max/min, etc.



Gilmer et al. Neural Message Passing for Quantum Chemistry. ICML 2017

MPNN Message Computation

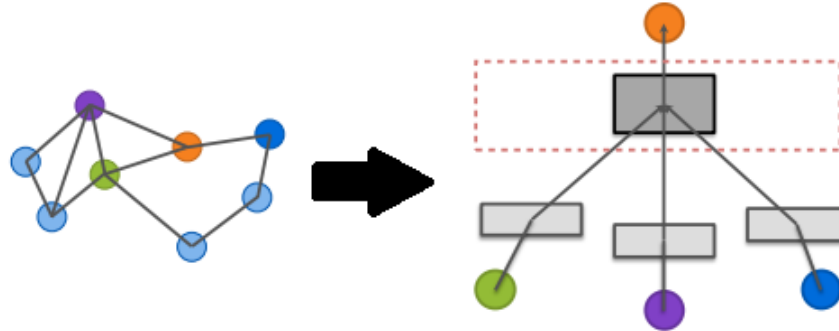
- Each node will create a message $m_{j \rightarrow i} = M^k(h_i^k, h_j^k, e_{ij})$
 - Dependent on incoming node (h_j^k), target node (h_i^k), edge feature
- e.g., Linear projection or FFN $\sigma(W_k \cdot [h_i^k, h_j^k, e_{ij}])$



Gilmer et al. Neural Message Passing for Quantum Chemistry. ICML 2017

Message Aggregation / Pooling

- Each node will aggregate messages from its neighbors
- Aggregate function can be:
 - Sum, Mean, Max operator
 - $m_i^{k+1} = \sum_{v_j \in N(v_i)} M^k(h_i^k, h_j^k, e_{ij})$



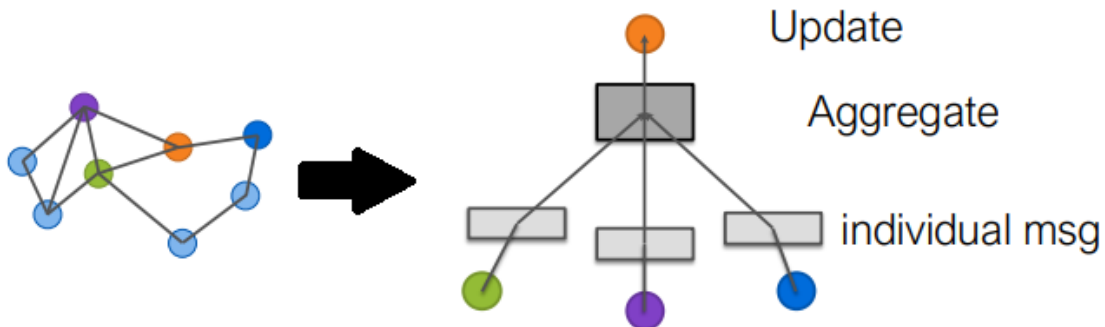
Gilmer et al. Neural Message Passing for Quantum Chemistry. ICML 2017

Message Update

- Combine message and current node embedding
- Apply FFN
 - $h_i^{k+1} = U^k(h_i^k, m_i^{k+1})$

Generic Framework: Message Passing NN

- MPNN = Message passing + Aggregation + Update
 - propagate nonlinear messages along graph topology
 - flexible and generic design choices under this framework
 - Graph convolutional network (GCN)
 - GraphSAGE
 - Graph Attention Network (GAT)



Gilmer et al. Neural Message Passing for Quantum Chemistry. ICML 2017

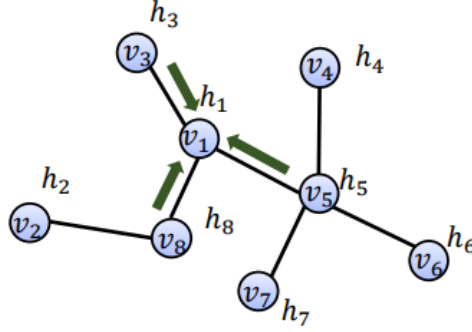
A Simple Graph Convolutional Network

- Averaging neighbor's message and apply nonlinear transformation

- Initial embedding: $h_i^0 = x_i$
- Computing next layer:

$$h_i^{k+1} = \sigma \left(W_k \frac{1}{|N(v_i)|} \sum_{v_j \in N(v_i)} h_j^k + B_k h_i^k \right)$$

Consider the example graph:



Gilmer et al. Neural Message Passing for Quantum Chemistry. ICML 2017

- Example: $h_1^2 = \tanh \left(W_1 \cdot \frac{1}{3} (h_3^1 + h_5^1 + h_8^1) + B_1 h_1^1 \right)$
- Next layer: $h_1^{(3)} = \tanh \left(W_2 \cdot \frac{1}{3} (h_3^{(2)} + h_5^{(2)} + h_8^{(2)}) + B_2 h_1^{(2)} \right)$

Matrix Representations of Graphs

We typically use an adjacency matrix:

- $A[i, j] = 1$ if v_i is adjacent to v_j .
- $A[i, j] = 0$, otherwise.
- Adjacency matrices for undirected graphs are symmetrical over the main diagonal.

Matrix Representation of GCN

- Neighbor Aggregation can be performed efficiently using matrix operations

$$H^k = [h_1^k, \dots, h_{|V|}^k]^T$$

- Then,

$$\sum_{v_j \in N(v_i)} h_j^k = A_{i,:} H^k$$

- Let D be diagonal matrix $D_{i,i} = \text{Degree}(v_i) = \sum_j A_{i,j}$
 - The Degree function returns the number of neighbors a given node has.

- Then,

$$\frac{1}{|N(v_i)|} \sum_{v_j \in N(v_i)} h_j^k = D^{-1} A H^k$$

Aggregation of Neighbor's Information in Matrix Form

Neighbor aggregation can be performed efficiently using matrix operations

$$\frac{1}{|N(v_i)|} \sum_{v_j \in N(v_i)} h_j^k = D^{-1} A H^k$$

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 4 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 4 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}^{-1} \cdot \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix} \cdot \begin{matrix} H^k \\ \text{Diagram of a graph with 8 nodes and edges, where the 2nd node is highlighted with a red dashed box.} \end{matrix}$$

Graph Convolution in Matrix Form

$$\begin{aligned} H^k &= [h_1^k, \dots, h_{|V|}^k]^T \\ \tilde{A} &= D^{-1} A \\ H^{k+1} &= \sigma(\tilde{A} H^k \cdot W_k^T + H^k B_k^T) \\ h_i^{k+1} &= \sigma \left(W_k \frac{1}{|N(v_i)|} \sum_{v_j \in N(v_i)} h_j^k + B_k h_i^k \right) \end{aligned}$$

(Better) Graph Convolutional Network

To make $\tilde{A}$ symmetric, we keep the definition of H^k :

$$H^k = [h_1^k, \dots, h_{|V|}^k]^T$$

But,

$$\tilde{A} = D^{-\frac{1}{2}} A D^{-\frac{1}{2}}$$

We also keep the same definition of H^{k+1} :

$$H^{k+1} = \sigma(\tilde{A} H^k \cdot W_k^T + H^k B_k^T)$$

Prediction Layer

To predict, we apply softmax functions

- For node classification, (e.g. predicting atom type)

$$o_i = \text{Softmax}(h_i^{(m)})$$

- For graph classification, (e.g., predicting toxicity)

$$o = \text{Softmax}\left(\frac{1}{N} \sum_i h_i^{(m)}\right)$$

Property of GCN: Equivariant

The embeddings computed from graph convolution layers is **invariant** to node order permutation.

Training GCN

- Parameters: weight matrix for each layer
- Supervised training: e.g., Node classification
 - Cross entropy loss

$$L = - \sum y_i \log f(h_i^K)$$

$$f_i = \text{Softmax}(h_i^{(K)})$$

- y_i is node label (in one-hot vector or smoothed version)
- Unsupervised training:
 - Linked nodes have similar embedding

$$L = \sum_{i,j} CE(y_{i,j}, \text{Sim}(h_i^K, h_j^K))$$

- $y_{i,j} = 1$ if there is an edge from v_i to v_j
 - Similarity can be defined in many ways: e.g., inner product $h_i \cdot h_j$

GraphSAGE

Also a special case of MPNN

$$h_i^{k+1} = \sigma(W_k \cdot \text{CONCAT}(h_i^k, \text{AGG}(\{h_j^k, \forall v_j \in N(v_i)\})))$$

AGG is the aggregate function and can be designed in multiple ways, like pooling (sum, avg, max).

Graph Attention Network (GAT)

Not strictly MPNN, but a generalized one:

$$h_i^{k+1} = \sigma \left(\sum_{v_j \in N(v_i)} \alpha_{ij} W_k h_{v_j}^k \right)$$

Attention weights:

$$\alpha_{ij} = \text{Attention}(W_k h_i, W_k h_j) = \frac{\exp(W_h h_i)^T W_k h_j}{\sum_{j'} \exp(W_h h_i)^T W_k h_{j'}}$$

Multi-head attention can also be used on GAT.

Relation between GNN and CNN

A CNN (convolutional neural network) can be viewed as a special GNN on a grid graph. Essentially, one node has 8 nodes surrounding it, four adjacent, four diagonal.

GNN vs. Transformer

A transformer is a special GNN on a fully-connected graph.

Equivariant Graph Neural Network

Structure and Geometry of Molecules

- Matrix of 3D coordinates, in addition to node features
- Equivariance: translation, rotation invariant

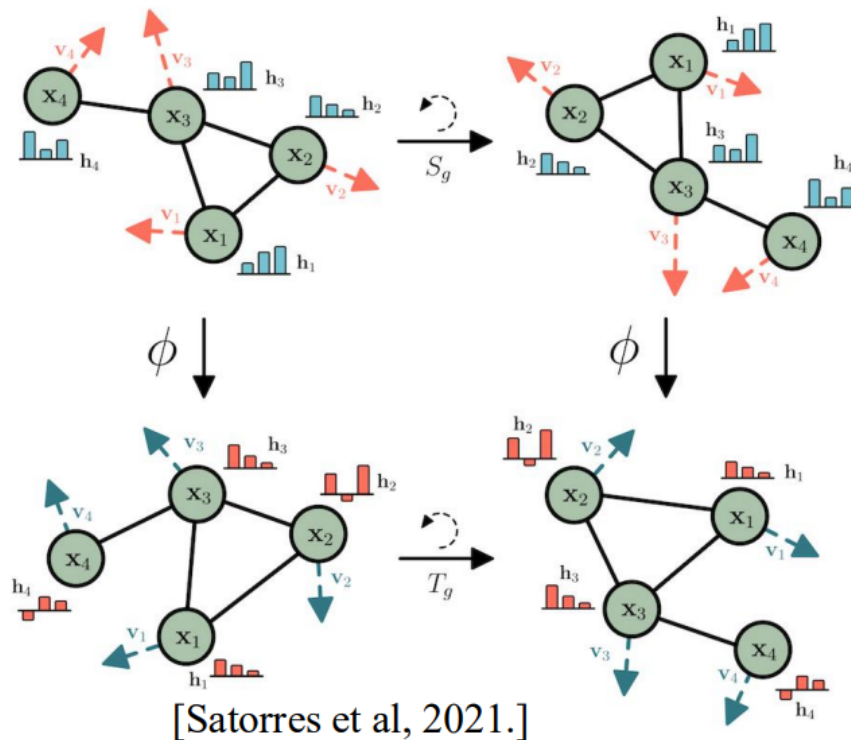
Equivariance in Embedding Transformation

- f is Equivariant (SE3 invariant):

$$f(R(x) + z) = f(x)$$

- No matter how we rotate and move the molecule, the NN will compute the same embeddings.
- X are coordinates
- H are features
- g is the rotation / translation.

$$F(H, g(X)) = F(H, X)$$



A **Equivariant Graph Neural Network (EGNN)** is an equivariant MPNN, optionally updating coordinates.

$$\begin{aligned}
 d_{ij} &= |x_i - x_j|_2 \\
 m_{j \rightarrow i} &= M^k(h_i^k, h_j^k, d_{ij}) \\
 m_i^{k+1} &= \sum_{v_j \in N(v_i)} m_{j \rightarrow i} \\
 h_i^{k+1} &= U^k(h_i^k, m_i^{k+1})
 \end{aligned}$$

Auto-Encoder

Learning representations so that data can be reconstructed! [FILL 4] Need to avoid learning an identity mapping

- Simple instance:
 - encoder is linear projection
 - decoder is linear projection
 - weight tying
- Other constraints:
 - Robust construction to noise, (Denoising Auto-Encoder)
 - Hidden representations conform to a distribution (Variational Auto-Encoder)

[Fill 5] The box with h is a probability function (to provide the variability for the Variational Auto-Encoder)

Training Auto-Encoder

- Real-valued data (e.g., images)
 - MSE loss
 - $MSE = ||f(x) - x||$
- Categorical data (e.g., sentences)
 - Cross-Entropy
 - $CE = - \sum x_i \log f(x)_i$

Denoising Auto-Encoder

- Encoder takes a noised data to produce the hidden representation
- Decoder needs to recover original data from hidden representation
- Make the auto-encoder robust

[Fill 9]

Ovoercomplete Representation

- If hidden representation is larger than input data dimension
- Need to enforce additional constraints (e.g., sparsity)

FILL loss

Variational Auto-Encoder (VAE)

- Input, intermediate, and output values are no longer deterministic vectors, but are probabilistic variables
- Using probabilistic graphical model (PRG) to represent dependencies

Probabilistic Graphical Model

- Models describe how the world works
- Models are always simplifications
 - May not account for every variable
 - May not account for all interactions between variables
- What do we do with probabilistic models?
 - We (or our agents) need to reason about unknown variables, given evidence
 - Example: explanation (diagnostic reasoning)
 - Example: prediction (causal reasoning)
 - Example: value of information

Directed PGM - Bayesian Networks

- One type of PGM with directed graph
- Nodes: random variables (with domains)
 - Can be assigned (observed) or unassigned (unobserved)
- Arcs: interactions
 - Indicate “direct influence” between variables
 - Formally: encode conditional independence (more later)
- For now: imagine that arrows mean direct causation (in general, they don’t!)

Bayesian Network Semantics

- A set of nodes, one per variable X
- A directed, acyclic graph (DAG)
- A conditional distribution for each node
 - A collection of distributions over X , one for each combination of parents’ values

$$P(X|a_1, \dots, a_n)$$

- CPT: conditional probability table
 - Description of a noisy “causal” table
- Directed graphical models

Example: Rain and Traffic

- Variables:
 - R : if it rains (1) otherwise (0)
 - T : if there is traffic (1) otherwise (0)
- Model 1: independence.
- Model 2: rain causes traffic [FILL tables + image]

Example: Traffic II

- Variables:
 - T: Traffic
 - R: It rains
 - U: Umbrella
- Q: What is the probability of rain if we observe traffic but no umbrella?
 - $P(R = 1|T = 1, U = 0)$

Variational Auto-Encoders

[FILL (before 20)]

Deep Latent Model

- z follows a prior distribution, e.g., $\text{Gaussian}(0, I)$
- $p(x|z)$ is defined by a deep neural network $f(z; \theta)$
 - We use a deep neural network to simulate the probability functions.
- To learn θ , use $E_{(Z|X)}[\log p(X, Z; \theta)]$

Probabilistic Graphical Model for VAE

- Assuming data X is generated from a latent variable Z
- Generation process:
 - draw $Z \sim N(\mu, \Sigma)$
 - draw $X|Z \sim p(f(Z))$, defined by a neural network f with parameter σ

[FILL 21]

Derivation of VAE

Objective: maximize the data log-likelihood

$$\begin{aligned}\max \ell(\theta) &= \sum_n \log p(x_n; \theta) \\ &= \sum_n \log \int p(x_n|z_n; \theta) p(z_n; \theta) \, dz_n\end{aligned}$$

- Hard to optimize over θ , if $f(Z)$ is very complex, due to the integration in the log. (i.e., $p(x_n|z_n; \theta)$ is very complex)
- Examples of complex $f(Z)$ include CNNs, RNNs, GNNs, Transformers

Variational Posterior Distribution

- See equations above
- $\log p(x; \theta)$ is intractable.
- For any distribution $q(z|x, \phi)$:

$$\log p(x; \theta) \geq E_{q(z|x; \phi)} \left[\log \frac{p(x, z; \theta)}{q(z|x; \phi)} \right] = \text{ELBO}$$

- Derivation via Jensen's inequality.
 - If we want to find $f(E(x))$, where $E(x)$ is an expectation (e.g., average value), and f is a *concave* function, then $E[f(x)] < f(E(x))$.
 - It turns out in the ELBO equation, we are essentially maximizing a lower bound for our equation.
- Maximizing the ELBO instead of maximizing $\log p(x; \theta)$

Understanding ELBO

$$\begin{aligned} \log p(X; \theta) &\geq E_q \left[\log \frac{p(X, Z; \theta)}{q(Z|X; \phi)} \right] \\ &= \sum_n E_q \left[\log \frac{p(x_n|z_n; \theta)p_0(z_n)}{q(z_n|x_n; \phi)} \right] \\ &= E_q [\log p(x_n|z_n; \theta)] - KL(q(z_n|x_n; \phi) || p_0(z_n)) \end{aligned}$$

- The first term (before subtraction) is reconstruction loss
- The second term is regularization. (KL = Kullback-Leibler divergence)

Variational Auto-Encoder with Neural Network

Let $q(z|x; \phi)$ and $p(x|z; \theta)$ share the same parameter θ [FILL 25]

Training VAE

- Gradient Descent

FILL

[FILL]

VAE for Sequence Data